

Client Integration Requirements

Third-party services that need the client's accounts, API keys or decisions to go live.

Prepared by **TEXOCORE**
Platform: **Peace** (textiles e-commerce)
Date: **27 July 2026**

The platform is built around each service via a swappable provider-adapter, so none of these block the rest of the system. The **storefront, cart, checkout (Cash on Delivery), orders, GST invoices, admin panel, in-app notifications, campaigns and reports all work today** without any of the items below. This sheet lists only what must come from the client's side.

- **Keys only** — code done, add credentials
- **Keys + wiring** — small code once provider chosen
- **Full build** — feature built once account is ready
- **Done** — set up already

Quick summary

#	INTEGRATION	WHAT THE CLIENT PROVIDES	READINESS
1	Razorpay — online payments	Razorpay account + API keys	KEYS ONLY
2	Amazon S3 — media storage	AWS bucket + access keys	KEYS ONLY
3	Email — order & marketing mail	Provider choice + credentials	KEYS + WIRING
4	SMS — OTP & order alerts	SMS provider + DLT sender	KEYS + WIRING
5	WhatsApp — updates & promos	WhatsApp Business API + templates	KEYS + WIRING
6	Push (FCM) — optional	Firebase Cloud Messaging keys	KEYS + WIRING
7	BlueDart — courier / shipping	BlueDart API account + license	FULL BUILD
8	Hosting — go-live	Hosts, domain, SSL	FULL BUILD
9	Firebase Auth — login	Confirm project ownership	DONE

Details

1 Payments — Razorpay

KEYS ONLY

Online card / UPI / netbanking payments. **Cash on Delivery works today without this.** Order creation, payment-signature verification and webhook verification are all implemented — add keys and online payments switch on automatically.

Client provides

KYC-approved Razorpay account (live mode)

Keys (backend)

RAZORPAY_KEY_ID

RAZORPAY_KEY_SECRET

RAZORPAY_WEBHOOK_SECRET

Refunds use the same account — a small gateway-refund call remains to wire (today cancel/return marks the order `REFUNDED` manually).

2 Media storage — Amazon S3

KEYS ONLY

Product images and videos. Today files save to local disk (fine for development); production needs durable storage with a CDN. S3 upload / delete is implemented — flip the provider and add credentials.

Client provides

AWS account + S3 bucket (optional CloudFront CDN)

Keys (backend)

AWS_S3_BUCKET

AWS_S3_REGION

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

AWS_S3_CDN_URL

Switch

MEDIA_PROVIDER=s3

Any S3-compatible store (Cloudflare R2, MinIO) works with the same keys.

3 Email service

KEYS + WIRING

Order confirmations, password reset, price-drop / cart / campaign emails. Today they log to the developer console, so every flow is testable. Once the client picks a provider, the send call is wired against it (~10–20 lines).

Client provides

Provider choice (Gmail SMTP, Brevo, Amazon SES, Zoho...) + credentials + verified sender

Keys (backend)

Provider credentials

EMAIL_FROM

EMAIL_PROVIDER=client

Free options exist (Gmail SMTP, Brevo free tier).

4 SMS service

KEYS + WIRING

OTP and order-status texts. Today: developer console. Adapter shell is ready; the send call is implemented against the chosen provider.

Client provides

SMS provider account (MSG91, Twilio...) + DLT sender registration (India)

Keys (backend)

MSG91_AUTH_KEY

MSG91_SENDER_ID

SMS_PROVIDER=client

India requires DLT template approval — a client / legal lead-time step.

5 WhatsApp service

KEYS + WIRING

Order updates and promotions on WhatsApp. Today: developer console. Adapter shell is ready; the send call is implemented once access and templates are approved.

Client provides

WhatsApp Business API access (Meta Cloud API, Gupshup...) + approved message templates

Keys (backend)

WHATSAPP_PHONE_NUMBER_ID

WHATSAPP_ACCESS_TOKEN

WHATSAPP_PROVIDER=client

Meta template approval is a client-side lead-time item.

6 Push notifications — FCM (optional)

KEYS + WIRING

Browser / app push. The in-app notification bell already works without this.

Client provides

Firebase Cloud Messaging server credentials

Switch

PUSH_PROVIDER=client

7 Courier / shipping — BlueDart

FULL BUILD

Serviceability by pincode, live rates, AWB / label generation, pickup scheduling and a live tracking timeline. **Today fulfilment status is set manually in admin** (packed → shipped → delivered). Config placeholders exist, but the courier adapter itself must be built — this is the largest remaining code item, not a config switch.

Client provides

BlueDart API account + license

Keys (backend)

BLUEDART_LICENSE_KEY

BLUEDART_LOGIN_ID

BLUEDART_API_BASE

COURIER_PROVIDER=bluedart

Any courier (Delhivery, Shiprocket...) can slot into the same adapter if the client prefers a different one.

8 Hosting & deployment

FULL BUILD

Going live. No application code needed — accounts and deployment configuration.

Client provides / decides

Frontend host (Vercel free tier works) · backend host + managed Postgres · domain name + SSL

Config

NEXT_PUBLIC_API_BASE_URL & CORS_ORIGINS set to production URLs

9 Firebase Auth

DONE

Customer and admin login (email-password + Google). Project `peace-texocore` is live on the free Spark plan and fully configured.

Client action

Confirm ownership of this Firebase project (or provide their own) and review the plan if usage grows

Keys

FIREBASE_* (backend)

NEXT_PUBLIC_FIREBASE_* (web)

Business data needed for go-live

Not integrations, but required to launch:

- **Official logo & brand assets** **PENDING FROM CLIENT**

Full-colour logo in vector (SVG) + PNG (light & dark background versions), a square icon / favicon, and brand colour codes. The logo shown in the app and on this document is a **placeholder** until the official artwork is supplied.

- GSTIN, HSN codes and legal company name / address for invoices

- Return, refund, privacy and shipping policy content

- Brand colours (theme is admin-editable once colours are confirmed)

- Support email and contact phone number
